

**LAPORAN TUGAS PEMROGRAMAN BERORIENTASI OBJEK
ANALISA KODE SISTEM INFORMASI TOKO BERBASIS JAVA**



Dosen Pengampu : Dr. Ni Made Satvika Iswari, S.T., M.T.

Disusun Oleh :

Nama : Putu Agus Saputra

NIM : 2501010063

Kelas : SI MLM1

Program Studi Sistem Informasi
Universitas Primakara

2026

DESKRIPSI TUGAS

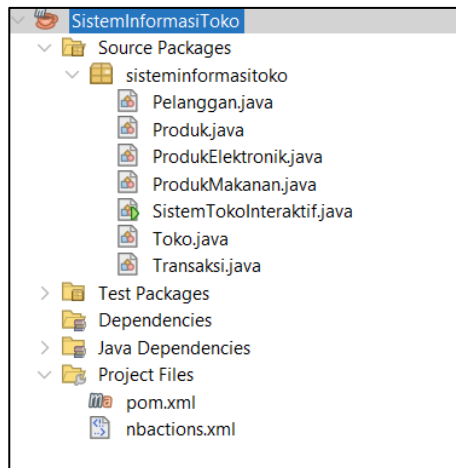
1. Perangkat yang digunakan

KOMPONEN	KETERANGAN
Sistem Operasi	Windows 11
IDE	Apache NetBeans IDE 31
JDK	JDK 26
Build Tool	Maven
Nama Proyek	SistemInformasiToko
Nama Package	sisteminformasitoko

2. Proses pembuatan Proyek

1. Membuat proyek baru : File > New Project > Java With Maven > java Application
2. Mengisis Project Name, Group Id, dan Package, lalu klik Finish.
3. Menghapus berkas **App.java** bawaan Maven agar tidak ada dua method main.
4. Memasukkan ketujuh file **.java** ke dalam package pada Source Packages.
5. Menambahkan baris **package** sisteminformasitoko; pada baris pertama setiap file.
6. Menyesuaikan versi Java pada **pom.xml** agar tidak melebihi **versi JDK** yang terpasang.
7. Mengatur Main Class pada Properties > Run menjadi **sisteminformasitoko.SistemTokoInteraktif**
8. Menjalankan Clean and Build, lalu Run File pada **SistemTokoInteraktif.java**.

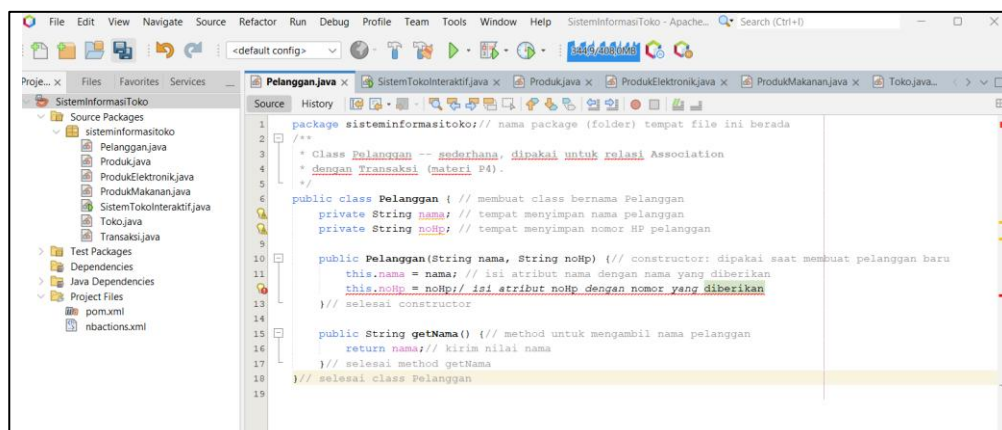
3. Tangkapan Layar dan Penjelasan



Gambar 3. 1 Struktur Proyek SistemInformasiToko

Penjelasan gambar 3.1 : Package dibuat terlebih dahulu karena package berfungsi sebagai folder yang mengelompokkan ketujuh file Java agar menjadi satu kesatuan aplikasi. Dengan satu package yang sama (sisteminformasitoko), setiap class dapat saling mengenali dan memanggil class lain tanpa perlu import tambahan, misalnya Transaksi langsung memakai Pelanggan dan Produk.

Package juga membuat kode lebih rapi dan terhindar dari bentrok nama jika ada class lain yang namanya sama. Selain itu, Maven dan NetBeans membutuhkan package untuk menemukan class utama, sehingga file yang diletakkan langsung di luar package sering menimbulkan error saat *Clean and Build* atau *Run*. Karena itu, baris package sisteminformasitoko; harus ada di baris pertama setiap file.



Gambar 3. 2 Kode Pelanggan.java

Penjelasan gambar 3.2 : Kode ini mendefinisikan Class Pelanggan menyimpan identitas pembeli lewat dua atribut bertipe String, yaitu

nama dan noHp, yang keduanya diberi modifier private agar hanya bisa dibaca lewat method resmi. Nama jelas berupa teks. Nomor HP sengaja memakai String, bukan int, karena nomor telepon bukan bilangan yang akan dihitung; angka nol di depan (mis. 0812...) akan hilang jika disimpan sebagai int, nomor bisa memuat tanda + atau -, dan panjangnya melebihi batas int.

Constructor mengisi kedua atribut saat objek dibuat, sedangkan getName() mengembalikan String agar Transaksi dapat menampilkan nama pembeli pada struk.

```

1 package sisteminformasi; // nama package (folder) tempat file ini berada
2 /**
3  * SISTEM INFORMASI WORD - Class Produk (superclass)
4  * Nama: <u>Akhmad Rizki</u> (P3) * <u>danar</u> <u>akab</u> <u>inheritas</u> (P5)
5  *
6  * Nama atribut dibuat PRIVATE -- ini prinsip Encapsulation: data pribadi
7  * (harga, stok) tidak boleh diubah sembarangan dari luar. harus lewat
8  * method <u>nama</u> yang <u>akab</u> <u>diikuti</u> ATURAN KETENTUAN (palingan).
9  */
10
11 public class Produk { // membuat class bernama Produk
12     private String nama; // tempat menyimpan nama produk
13     private double harga; // tempat menyimpan harga produk
14     private int stok; // tempat menyimpan jumlah stok produk
15
16     public Produk(String nama, double harga, int stok) { // constructor: dipakai saat membuat produk baru
17         this.nama = nama; // ini nama produk
18         this.harga = harga; // ini harga produk
19         this.stok = stok; // ini stok produk
20         // selesai constructor produk
21     }
22
23     public String getName() { // method untuk mengambil nama produk
24         return nama; // kirim nilai nama
25     }
26
27     public double getHarga() { // method untuk mengambil harga produk
28         return harga; // kirim nilai harga
29     }
30
31     public int getStok() { // method untuk mengambil stok produk
32         return stok; // kirim nilai stok
33     }
34
35     // ATURAN BLOK: harga tidak boleh diubah jadi negatif
36     public void setHarga(double hargaBaru) { // method untuk mengubah harga
37         if (hargaBaru >= 0) { // cek: harga baru harus 0 atau lebih
38             harga = hargaBaru; // harga benar, jadi harga diganti
39         } else { // kalau harga baru negatif
40             System.out.println("Cagal: harga tidak boleh negatif"); // tampilkan pesan gagal
41         }
42     }
43
44     // ATURAN KETENTUAN: stok tidak boleh berkurang melebihi yang tersedia
45     public boolean kurangStok(int jumlah) { // method untuk mengurangi stok hasilnya true atau false
46         if (jumlah > 0 && jumlah <= stok) { // cek: jumlah lebih dari 0 dan tidak melebihi stok
47             stok = jumlah; // kurangi stok sebanyak jumlah
48             return true; // Kirim true artinya berhasil
49         }
50         return false; // Kirim false artinya gagal
51     }
52
53     public void tambahStok(int jumlah) { // method untuk menambah stok
54         if (jumlah > 0) { // cek: jumlah harus lebih dari 0
55             stok += jumlah; // tambah stok sebanyak jumlah
56         }
57     }
58
59     // method ini akan di-override oleh subclass (ProdukElektronik, ProdukMakanan, ProdukKecantikan)
60     public String tampilInfo() { // method untuk membuat teks info produk
61         return nama + " | Rp" + harga + " | Stok: " + stok; // gabungkan nama, harga, dan stok jadi satu teks
62     }
63
64     // selesai class Produk
65 }

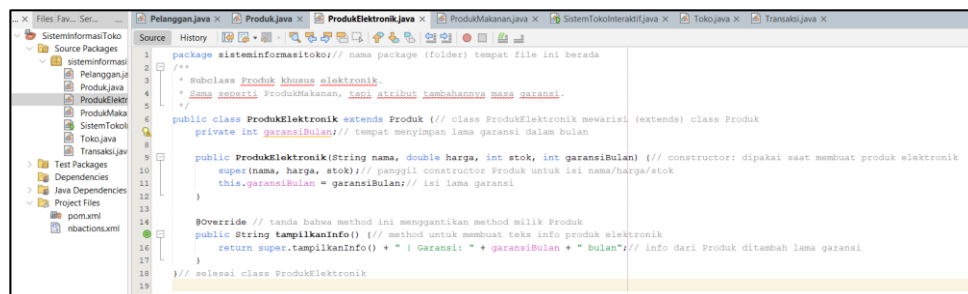
```

Gambar 3. 3 kode produk.java

Penjelasan gambar 3.3 : Kode ini mendefinisikan Class Produk menyimpan tiga data dengan tipe yang disesuaikan sifatnya. nama bertipe String karena berupa teks. harga bertipe double karena harga bisa memiliki pecahan dan akan dikalikan dengan jumlah beli; int tidak cocok karena membuang angka desimal. (Pada aplikasi keuangan sungguhan

BigDecimal lebih akurat, tetapi double sudah cukup untuk tugas ini.) stok bertipe int karena jumlah barang selalu bilangan bulat.

Ketiga atribut bersifat private, sehingga perubahan harus lewat method yang punya aturan bisnis. setHarga() bertipe void karena hanya mengubah nilai dan menolak harga negatif. kurangiStok() bertipe boolean karena hasilnya hanya dua kemungkinan, berhasil (true) atau gagal (false), sehingga pemanggil langsung tahu apakah stok cukup. tampilkanInfo() bertipe String karena mengembalikan teks gabungan nama, harga, dan stok, dan method inilah yang ditimpa oleh subclass.



```

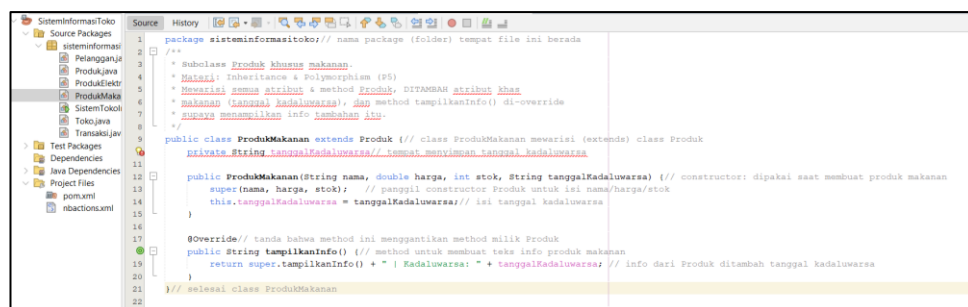
1 package sisteminformasitoko; // nama package (folder) tempat file ini berada
2 /**
3  * Subclass Produk khusus elektronik.
4  * Nama, harga, stok, dan atribut tambahannya masa garansi.
5  */
6 public class ProdukElektronik extends Produk { // class ProdukElektronik mewarisi (extends) class Produk
7     private int garansiBulan; // tempat menyimpan lama garansi dalam bulan
8
9     public ProdukElektronik(String nama, double harga, int stok, int garansiBulan) { // constructor: dipakai saat membuat produk elektronik
10         super(nama, harga, stok); // panggil constructor Produk untuk isi nama/harga/stok
11         this.garansiBulan = garansiBulan; // isi lama garansi
12     }
13
14     @Override // tanda bahwa method ini menggantikan method milik Produk
15     public String tampilkanInfo() { // method untuk membuat teks info produk elektronik
16         return super.tampilkanInfo() + " | Garansi: " + garansiBulan + " bulan"; // info dari Produk ditambah lama garansi
17     }
18 } // selesai class ProdukElektronik

```

Gambar 3. 4 kode produk elektronik.java

Penjelasan gambar 3.4 : Kode Class ini juga mewarisi Produk dan menambah atribut garansiBulan. Tipenya int karena masa garansi dinyatakan dalam bilangan bulat bulan (6, 12, 24) tanpa pecahan, dan satuan "bulan" dituliskan pada nama atribut agar maksudnya jelas.

constructor meneruskan tiga data dasar lewat super, dan tampilkanInfo() yang bertipe String menambahkan keterangan garansi, sehingga hasil tampilan berbeda untuk tiap jenis produk (polymorphism).



```

1 package sisteminformasitoko; // nama package (folder) tempat file ini berada
2 /**
3  * Subclass Produk khusus makanan.
4  * Mewarisi: Inheritance & Polymorphism (PS)
5  * Mewarisi semua atribut & method Produk, DITAMBAH atribut khas
6  * makanan (tanggal kadaluarsa), dan method tampilkanInfo() di-override
7  * supaya menampilkan info tambahan itu.
8  */
9 public class ProdukMakanan extends Produk { // class ProdukMakanan mewarisi (extends) class Produk
10     private String tanggalKadaluarsa; // tempat menyimpan tanggal kadaluarsa
11
12     public ProdukMakanan(String nama, double harga, int stok, String tanggalKadaluarsa) { // constructor: dipakai saat membuat produk makanan
13         super(nama, harga, stok); // panggil constructor Produk untuk isi nama/harga/stok
14         this.tanggalKadaluarsa = tanggalKadaluarsa; // isi tanggal kadaluarsa
15     }
16
17     @Override // tanda bahwa method ini menggantikan method milik Produk
18     public String tampilkanInfo() { // method untuk membuat teks info produk makanan
19         return super.tampilkanInfo() + " | Kadaluarsa: " + tanggalKadaluarsa; // info dari Produk ditambah tanggal kadaluarsa
20     }
21 } // selesai class ProdukMakanan

```

Gambar 3. 5 kode produk makanan.java

Penjelasan gambar 3.5 : Dengan extends, ProdukMakanan mewarisi seluruh data dan method Produk, sehingga hanya perlu menambahkan satu atribut khas, yaitu tanggalKadaluwarsa. Tipenya String karena pada program ini tanggal hanya ditampilkan sebagai teks dan tidak dihitung, jadi format seperti "25-09-2026" sudah cukup.

Constructor memakai super(...) untuk mengirim nama, harga, dan stok ke Produk, dan tampilkanInfo() yang bertipe String di-override untuk menambahkan keterangan kadaluwarsa di belakang info dasar.

```

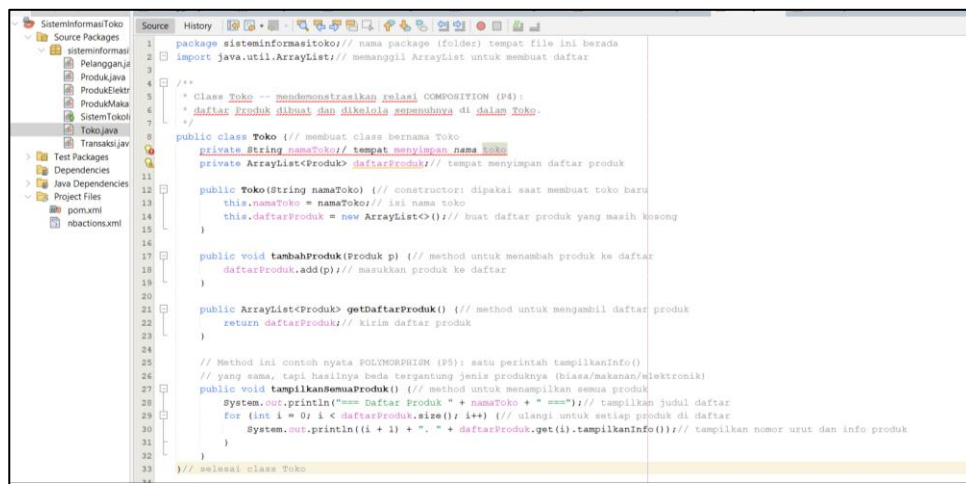
1  /**
2   * SISTEM INFORMASI TOKO - Program Utama (Interaktif)
3   * Menggabungkan semua materi Pertemuan 3-5:
4   * - Enkapsulasi : atribut Produk private + validasi harga/stok
5   * - Class Diagram : relasi association (Transaksi) & composition (Toko)
6   * - Inheritance & Polymorphism : ProdukMakanan/ProdukElektronik men-override tampilkanInfo()
7   *
8   * Program ini SENGAJA dibuat sederhana supaya bisa jadi PONDASI project
9   * kalian -- nanti tinggal ditambah GUI (Pertemuan 12) dan database (Pertemuan 14).
10  */
11
12  public class SistemTokoInteraktif { // class utama untuk menjalankan sistem toko
13
14      public static void main(String[] args) { // method utama yang pertama dijalankan
15          try {Scanner input = new Scanner(System.in); // membuat Scanner untuk membaca input keyboard
16              Toko toko = new Toko("KD_Prime_Mart"); // membuat objek toko dengan nama KD_Prime_Mart
17
18              // ===== Data awal (boleh diubah/ditambah sendiri) =====
19              toko.tambahProduk(new ProdukMakanan("Roti Tawar", 15000, 20, "25-09-2026")); // menambahkan produk makanan
20              toko.tambahProduk(new ProdukElektronik("Kabel USB-C", 45000, 15, 6)); // menambahkan produk elektronik
21              toko.tambahProduk(new Produk("Buku Tulis", 5000, 50)); // menambahkan produk biasa
22              toko.tambahProduk(new Produk("Stopkontak", 65000, 20)); // menambahkan produk elektronik
23              toko.tambahProduk(new Produk("FiberCloth", 35000, 15)); // menambahkan produk biasa
24
25              System.out.println("==== SELAMAT DATANG DI SISTEM INFORMASI KD_Prime_Mart ==="); // tampilkan ucapan selamat datang
26              System.out.print("Masukkan nama Anda: "); // minta pengguna mengetik nama
27              String namaPembeli = input.nextLine(); // baca nama yang diketik
28              Pelanggan pelanggan = new Pelanggan(namaPembeli, "-"); // buat pelanggan baru
29
30              boolean lanjut = true; // tanda program masih berjalan
31              while (lanjut) { // ulang menu selama lanjut bernilai true
32                  System.out.println(); // tampilkan baris kosong
33                  toko.tampilkanSemuaProduk(); // <- di sini lah POLYMORPHISM kelihatan hasilnya
34                  System.out.println(toko.getDaftarProduk().size() + 1) + ". Keluar"; // tampilkan menu Keluar
35                  System.out.print("Pilih nomor produk yang ingin dibeli: "); // minta pengguna memilih nomor
36                  int pilihan = Integer.parseInt(input.nextLine()); // membaca pilihan produk dari pengguna
37
38                  if (pilihan == toko.getDaftarProduk().size() + 1) { // mengecek apakah pengguna memilih keluar
39                      lanjut = false; // hentikan perulangan
40                  } else if (pilihan >= 1 && pilihan <= toko.getDaftarProduk().size()) { // mengecek apakah pilihan produk valid
41                      Produk produkDipilih = toko.getDaftarProduk().get(pilihan - 1); // mengambil produk berdasarkan pilihan
42                      System.out.print("Jumlah beli: "); // meminta jumlah produk yang ingin dibeli
43                      int jumlah = Integer.parseInt(input.nextLine()); // membaca jumlah pembelian
44
45                      if (produkDipilih.kurangStok(jumlah)) { // mengecek dan mengurangi stok produk
46                          Transaksi transaksi = new Transaksi(pelanggan, produkDipilih, jumlah); // membuat objek transaksi
47                          System.out.println(); // membuat baris kosong
48                          transaksi.cetakStruk(); // menampilkan struk transaksi
49                      } else {
50                          System.out.println(">>> Maaf, stok tidak mencukupi."); // menampilkan pesan jika stok kurang
51                      }
52                  } else {
53                      System.out.println(">>> Pilihan tidak valid."); // menampilkan pesan jika pilihan salah
54                  }
55
56                  System.out.println("Terima kasih, " + pelanggan.getNama() + "!"); // menampilkan ucapan terima kasih
57              }
58          }
59      }
60  }
61  // menutup Scanner setelah program selesai
62
63  }

```

Gambar 3.6 Kode SistemTokoInteraktif.java

Penjelasan gambar 3.6 : Program ini memakai beberapa tipe data sesuai kebutuhannya. Scanner dipakai untuk membaca ketikan keyboard. namaPembeli bertipe String karena berupa teks, sedangkan pilihan dan jumlah bertipe int karena berupa nomor menu dan banyak barang yang selalu bilangan bulat. Karena nextLine() selalu menghasilkan String, hasilnya diubah menjadi angka dengan Integer.parseInt. Cara ini dipilih daripada nextInt() supaya tidak ada sisa baris baru (Enter) yang membuat input berikutnya terlewat.

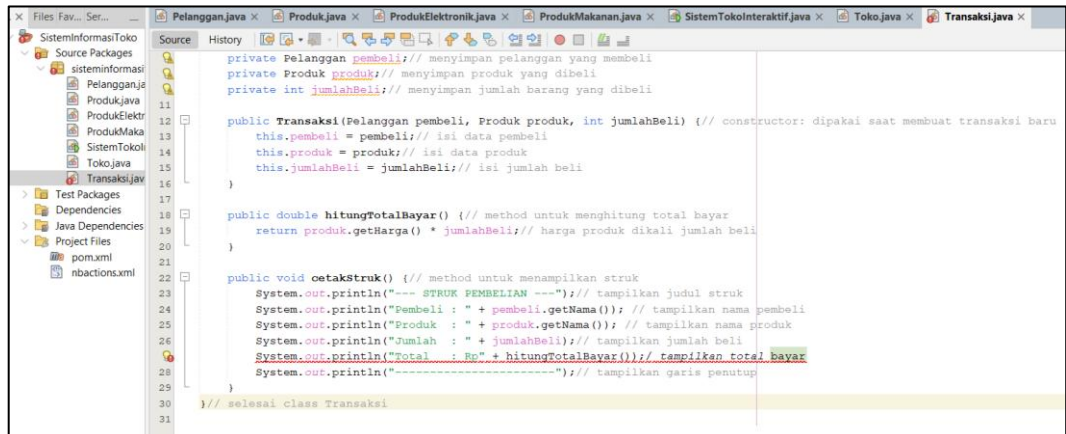
Variabel lanjut bertipe boolean karena hanya perlu dua keadaan, terus berjalan (true) atau berhenti (false), sehingga pas menjadi penentu perulangan while. Objek Toko, Pelanggan, dan Transaksi dibuat dengan new, sedangkan produk yang dipilih disimpan pada variabel bertipe Produk sehingga boleh berisi makanan, elektronik, maupun produk biasa. Hasil kurangiStok() yang bertipe boolean langsung dipakai sebagai syarat if untuk menentukan apakah struk dicetak atau muncul pesan stok kurang.



Gambar 3. 7 Kode Toko.java

Penjelasan gambar 3.7 : Class Toko menyimpan namaToko bertipe String dan daftarProduk bertipe ArrayList<Produk>. ArrayList dipilih daripada array biasa karena ukurannya bertambah otomatis setiap produk ditambahkan, sedangkan array harus ditentukan ukurannya sejak awal. Tanda <Produk> membatasi isi daftar pada objek Produk, dan karena ProdukMakanan serta ProdukElektronik adalah turunannya, semua jenis produk bisa masuk ke daftar yang sama. Daftar ini dibuat di dalam constructor Toko, sehingga produk dikelola oleh toko itu sendiri (composition).

tampilkanSemuaProduk() bertipe void karena hanya mencetak ke layar. Variabel i bertipe int karena berfungsi sebagai nomor urut sekaligus indeks list. Pemanggilan tampilkanInfo() pada tiap produk menghasilkan tampilan yang menyesuaikan jenis objeknya (polymorphism).



Gambar 3. 8 Kode transaksi.java

Penjelasan gambar 3.8: Class Transaksi mencatat pembelian dengan tiga atribut. Atribut pembeli bertipe Pelanggan dan produk bertipe Produk. Tipe Pelanggan dan Produk adalah class buatan sendiri, sedangkan pembeli dan produk adalah nama variabel yang menyimpan data pelanggan dan data barang pada transaksi tersebut. Dengan begitu, Transaksi dapat mengambil nama dan harga lewat getNama() dan getHarga(). Atribut jumlahBeli bertipe int karena banyak barang selalu bilangan bulat. Method hitungTotalBayar() bertipe double karena hasilnya harga (double) dikali jumlah beli, sedangkan cetakStruk() bertipe void karena hanya menampilkan struk ke layar.

LAMPIRAN : TAUTAN REPOSITORI GITHUB

Keterangan	Link
Link Repositori	https://github.com/agussaputra181/sistem_informasi_toko-
Link Riwayat commit	https://github.com/agussaputra181/sistem_informasi_toko-/commits/main/
Link Rilis	https://github.com/agussaputra181/sistem_informasi_toko-/releases/tag/v1.0